

```

344         if f.get("field") not in
criteria_updates["remove_filters"]
345     ]
346
347     # Add scoring rules
348     if "add_scoring_rules" in criteria_updates:
349         current_criteria.setdefault("scoring_rules",
[]).extend(
350             criteria_updates["add_scoring_rules"]
351         )
352
353     # Update database
354     await self.db.execute("""
355         UPDATE segmentations
356         SET criteria = $1, updated_at = NOW()
357         WHERE id = $2
358     """, current_criteria, segmentation_id)
359
360     # Log user feedback
361     await self.db.execute("""
362         INSERT INTO user_feedback
363         (user_id, feedback_type, entity_type, entity_id,
feedback_data)
364         VALUES ($1, 'segmentation_criteria', 'segmentation',
$2, $3)
365     """, user_id, segmentation_id, criteria_updates)
366
367     return {
368         "segmentation_id": segmentation_id,
369         "updated_criteria": current_criteria,
370         "changes_applied": criteria_updates
371     }

```

Usage in Agent:

Code block

```

1  # Evaluate contact against segment
2  result = await self.use_skill("segmentation_logic",
"evaluate_contact_fit",
3      contact=contact_data,
4      segmentation_id="segment-uuid"
5  )
6
7  # result = {
8  #     "fit_score": 85,
9  #     "score_breakdown": {
10 #         "title_match": 90,
11 #         "company_size_match": 80,
12 #         "user_defined_score": 75
13 #     },
14 #     "passes_filters": True
15 # }
16
17 # Update criteria based on user input
18 updated = await self.use_skill("segmentation_logic",
"update_segmentation_criteria",
19     segmentation_id="segment-uuid",
20     user_id="user-uuid",
21     criteria_updates={
22         "add_filters": [

```

```

23         {
24             "field":
25             "profile.confirmed_facts.budget_timeline.budget_available",
26             "operator": ">=",
27             "value": 100000,
28             "reason": "Raised minimum budget threshold"
29         }
30     ],
31     "add_scoring_rules": [
32         {
33             "field": "profile.ai_insights.buying_signals",
34             "operator": "length",
35             "value": {">": 3},
36             "points": 20,
37             "reason": "Prioritize high-intent prospects"
38         }
39     ]
40 )

```

2.3 Skill: FeedbackCaptureSkill

Purpose: Capture user feedback and learn from corrections

File: `skills/feedback_capture_skill.py`

Code block

```

1  from agent_sdk import Skill, tool
2  from typing import Dict, Any, List
3
4  class FeedbackCaptureSkill(Skill):
5      """
6      Capture and apply user feedback
7
8      Feedback types:
9      - Score adjustments (user overrides AI score)
10     - Insight validations (confirm/reject AI insights)
11     - Segmentation logic updates
12     - Contact classifications
13     - Relationship strength overrides
14     """
15
16     name = "feedback_capture"
17     description = "Capture user feedback for continuous learning"
18
19     def __init__(self, db_pool):
20         self.db = db_pool
21
22     @tool
23     async def capture_score_adjustment(
24         self,
25         user_id: str,
26         contact_id: str,
27         field_path: str,
28         original_score: int,
29         adjusted_score: int,
30         reason: str
31     ) -> Dict:
32         """

```

```

33         User manually adjusts a score
34
35     Example:
36         User changes relationship_strength from 75 → 90
37         Reason: "Just closed a big deal with them"
38
39     Args:
40         user_id: User making adjustment
41         contact_id: Contact being adjusted
42         field_path: JSONB path (e.g.,
"relationship.strength_score")
43         original_score: AI-generated score
44         adjusted_score: User's score
45         reason: Why user made this change
46
47     Returns:
48         Feedback record
49     """
50
51     feedback_data = {
52         "field_path": field_path,
53         "original_score": original_score,
54         "adjusted_score": adjusted_score,
55         "reason": reason,
56         "adjustment_delta": adjusted_score - original_score
57     }
58
59     # Store feedback
60     feedback_id = await self.db.fetchval("""
61         INSERT INTO user_feedback
62         (user_id, feedback_type, entity_type, entity_id,
feedback_data, applied)
63         VALUES ($1, 'score_adjustment', 'contact', $2, $3,
false)
64         RETURNING id
65     """, user_id, contact_id, feedback_data)
66
67     # Apply adjustment to contact
68     await self._apply_score_adjustment(contact_id, field_path,
adjusted_score)
69
70     # Mark as applied
71     await self.db.execute("""
72         UPDATE user_feedback
73         SET applied = true, applied_at = NOW()
74         WHERE id = $1
75     """, feedback_id)
76
77     # Learn from adjustment (update ML model parameters)
78     await self._learn_from_adjustment(feedback_data)
79
80     return {
81         "feedback_id": feedback_id,
82         "applied": True,
83         "learning_triggered": True
84     }
85
86     async def _apply_score_adjustment(
87         self,
88         contact_id: str,
89         field_path: str,
90         new_score: int

```

```

91         ):
92             """
93             Update contact profile with new score
94             """
95             # Build JSONB update query
96             path_parts = field_path.split(".")
97
98             await self.db.execute(f"""
99                 UPDATE contacts
100                 SET profile = jsonb_set(
101                     profile,
102                     '{{{"", ".join(path_parts)}}}',
103                     to_jsonb($1::int),
104                     true
105                 ),
106                 updated_at = NOW()
107                 WHERE id = $2
108             """, new_score, contact_id)
109
110     async def _learn_from_adjustment(self, feedback_data: Dict):
111         """
112         Learn from user adjustment
113
114         Examples of learning:
115         - User consistently adjusts scores higher for certain
116         titles
117             → Increase title_match weight for this user
118         - User adjusts scores lower for certain industries
119             → Decrease industry_match weight
120
121         # Store in learning database for later ML model training
122         # This is for future ML improvements
123         """
124         pass
125
126     @tool
127     async def capture_insight_validation(
128         self,
129         user_id: str,
130         contact_id: str,
131         insight_type: str,
132         insight_text: str,
133         validation: str, # "confirmed" | "rejected"
134         confirmed_data: Optional[Dict] = None
135     ) -> Dict:
136         """
137         User validates or rejects an AI insight
138
139         Example:
140         AI said: "Pain point: Manual data entry (85%
141         confidence)"
142         User confirms: "Yes, they spend 15 hrs/week on manual
143         work"
144
145         Args:
146         user_id: User validating
147         contact_id: Contact with insight
148         insight_type: "pain_point" | "goal" | "objection"
149         insight_text: Original AI-generated text
150         validation: "confirmed" or "rejected"
151         confirmed_data: Full data if confirmed

```

```

150     Returns:
151         Updated contact with moved/removed insight
152     """
153
154     feedback_data = {
155         "insight_type": insight_type,
156         "insight_text": insight_text,
157         "validation": validation,
158         "confirmed_data": confirmed_data
159     }
160
161     # Store feedback
162     feedback_id = await self.db.fetchval("""
163         INSERT INTO user_feedback
164         (user_id, feedback_type, entity_type, entity_id,
165 feedback_data, applied)
166         VALUES ($1, 'insight_validation', 'contact', $2, $3,
167 false)
168         RETURNING id
169 """, user_id, contact_id, feedback_data)
170
171     # Fetch contact
172     contact = await self.db.fetchrow(
173         "SELECT profile FROM contacts WHERE id = $1",
174         contact_id
175     )
176
177     profile = contact["profile"]
178
179     # Move insight from ai_insights to confirmed_facts (or
180     # remove if rejected)
181     if validation == "confirmed":
182         result = await self._move_insight_to_confirmed(
183             contact_id,
184             profile,
185             insight_type,
186             insight_text,
187             confirmed_data
188         )
189     else:
190         result = await self._remove_insight(
191             contact_id,
192             profile,
193             insight_type,
194             insight_text
195         )
196
197     # Mark feedback as applied
198     await self.db.execute("""
199         UPDATE user_feedback
200         SET applied = true, applied_at = NOW()
201         WHERE id = $1
202 """, feedback_id)
203
204     # Track accuracy for learning
205     await self._track_insight_accuracy(
206         insight_type,
207         validation,
208         confirmed_data
209     )
210
211     return {

```

```

209         "feedback_id": feedback_id,
210         "insight_moved": validation == "confirmed",
211         "insight_removed": validation == "rejected",
212         **result
213     }
214
215     async def _move_insight_to_confirmed(
216         self,
217         contact_id: str,
218         profile: Dict,
219         insight_type: str,
220         insight_text: str,
221         confirmed_data: Dict
222     ) -> Dict:
223         """
224         Move AI insight to confirmed facts
225         """
226
227         # Find and remove from ai_insights
228         ai_insights = profile.get("ai_insights", {})
229         suspected_key = f"suspected_{insight_type}s"
230         suspected_list = ai_insights.get(suspected_key, [])
231
232         matching_insight = None
233         for insight in suspected_list:
234             if insight.get(insight_type) == insight_text:
235                 matching_insight = insight
236                 suspected_list.remove(insight)
237             break
238
239         # Add to confirmed_facts
240         confirmed_facts = profile.get("confirmed_facts", {})
241         confirmed_key = f"{insight_type}s"
242
243         if confirmed_key not in confirmed_facts:
244             confirmed_facts[confirmed_key] = []
245
246         confirmed_entry = {
247             **confirmed_data,
248             "originally_detected_by_ai": True,
249             "ai_confidence": matching_insight.get("confidence") if
matching_insight else None,
250             "confirmed_at": "NOW()"
251         }
252
253         confirmed_facts[confirmed_key].append(confirmed_entry)
254
255         # Update validation tracking
256         insight_validation = profile.get("insight_validation", {})
257         if "confirmed_insights" not in insight_validation:
258             insight_validation["confirmed_insights"] = []
259
260         insight_validation["confirmed_insights"].append({
261             "original_ai_insight": insight_text,
262             "ai_confidence": matching_insight.get("confidence") if
matching_insight else None,
263             "confirmed_at": "NOW()",
264             "accuracy": "Correct"
265         })
266
267         # Update contact
268         profile["ai_insights"] = ai_insights

```

```

269     profile["confirmed_facts"] = confirmed_facts
270     profile["insight_validation"] = insight_validation
271
272     await self.db.execute("""
273         UPDATE contacts
274         SET profile = $1, updated_at = NOW()
275         WHERE id = $2
276     """, profile, contact_id)
277
278     return {"status": "moved_to_confirmed"}
279
280 async def _remove_insight(
281     self,
282     contact_id: str,
283     profile: Dict,
284     insight_type: str,
285     insight_text: str
286 ) -> Dict:
287     """
288     Remove rejected AI insight
289     """
290
291     # Remove from ai_insights
292     ai_insights = profile.get("ai_insights", {})
293     suspected_key = f"suspected_{insight_type}s"
294     suspected_list = ai_insights.get(suspected_key, [])
295
296     matching_insight = None
297     for insight in suspected_list:
298         if insight.get(insight_type) == insight_text:
299             matching_insight = insight
300             suspected_list.remove(insight)
301             break
302
303     # Add to rejected_insights
304     insight_validation = profile.get("insight_validation", {})
305     if "rejected_insights" not in insight_validation:
306         insight_validation["rejected_insights"] = []
307
308     insight_validation["rejected_insights"].append({
309         "original_ai_insight": insight_text,
310         "ai_confidence": matching_insight.get("confidence") if
matching_insight else None,
311         "rejected_at": "NOW()",
312         "reason": "User rejected"
313     })
314
315     # Update contact
316     profile["ai_insights"] = ai_insights
317     profile["insight_validation"] = insight_validation
318
319     await self.db.execute("""
320         UPDATE contacts
321         SET profile = $1, updated_at = NOW()
322         WHERE id = $2
323     """, profile, contact_id)
324
325     return {"status": "removed"}
326
327 async def _track_insight_accuracy(
328     self,
329     insight_type: str,

```

```

330         validation: str,
331         confirmed_data: Optional[Dict]
332     ):
333         """
334         Track AI insight accuracy for model improvement
335         """
336         # Store metrics for ML model retraining
337         pass
338
339     @tool
340     async def capture_custom_fact(
341         self,
342         user_id: str,
343         contact_id: str,
344         fact_type: str, # "pain_point" | "goal" | "requirement" |
        "preference"
345         fact_data: Dict
346     ) -> Dict:
347         """
348         User manually adds a fact about contact
349
350         Example:
351             User adds: "They need solution by Q1 2025" (goal)
352             User adds: "Prefers email over calls" (preference)
353
354         This goes directly to confirmed_facts (not AI-generated)
355         """
356
357         feedback_data = {
358             "fact_type": fact_type,
359             "fact_data": fact_data,
360             "source": "user_input"
361         }
362
363         # Store feedback
364         feedback_id = await self.db.fetchval("""
365             INSERT INTO user_feedback
366             (user_id, feedback_type, entity_type, entity_id,
feedback_data, applied)
367             VALUES ($1, 'custom_fact', 'contact', $2, $3, false)
368             RETURNING id
369         """, user_id, contact_id, feedback_data)
370
371         # Add to confirmed_facts
372         contact = await self.db.fetchrow(
373             "SELECT profile FROM contacts WHERE id = $1",
374             contact_id
375         )
376
377         profile = contact["profile"]
378         confirmed_facts = profile.get("confirmed_facts", {})
379
380         fact_key = f"{fact_type}s"
381         if fact_key not in confirmed_facts:
382             confirmed_facts[fact_key] = []
383
384         fact_entry = {
385             **fact_data,
386             "user_added": True,
387             "added_by": user_id,
388             "added_at": "NOW()"
389         }

```

```

390
391     confirmed_facts[fact_key].append(fact_entry)
392     profile["confirmed_facts"] = confirmed_facts
393
394     await self.db.execute("""
395         UPDATE contacts
396         SET profile = $1, updated_at = NOW()
397         WHERE id = $2
398     """, profile, contact_id)
399
400     # Mark as applied
401     await self.db.execute("""
402         UPDATE user_feedback
403         SET applied = true, applied_at = NOW()
404         WHERE id = $1
405     """, feedback_id)
406
407     return {
408         "feedback_id": feedback_id,
409         "fact_added": True,
410         "fact_type": fact_type
411     }
412
413 @tool
414 async def get_feedback_summary(
415     self,
416     user_id: str,
417     days: int = 30
418 ) -> Dict:
419     """
420     Get summary of user feedback for learning
421
422     Returns insights like:
423     - Most frequently adjusted scores
424     - Insight validation accuracy
425     - Common custom facts added
426     """
427
428     query = """
429         SELECT
430             feedback_type,
431             COUNT(*) as count,
432             feedback_data
433         FROM user_feedback
434         WHERE user_id = $1
435             AND created_at > NOW() - INTERVAL '%s days'
436         GROUP BY feedback_type, feedback_data
437         ORDER BY count DESC
438     """ % days
439
440     rows = await self.db.fetch(query, user_id)
441
442     summary = {
443         "total_feedback_count": sum(row["count"] for row in
rows),
444         "by_type": {}
445     }
446
447     for row in rows:
448         feedback_type = row["feedback_type"]
449         if feedback_type not in summary["by_type"]:
450             summary["by_type"][feedback_type] = {

```

```
451         "count": 0,
452         "examples": []
453     }
454
455     summary["by_type"][feedback_type]["count"] +=
row["count"]
456     summary["by_type"][feedback_type]
["examples"].append(row["feedback_data"])
457
458     return summary
```

Usage in Agent:

Code block

```
1  # User adjusts score
2  await self.use_skill("feedback_capture",
    "capture_score_adjustment",
3      user_id="user-uuid",
4      contact_id="contact-uuid",
5      field_path="relationship.strength_score",
6      original_score=75,
7      adjusted_score=90,
8      reason="Just closed a deal with them, relationship is stronger"
9  )
10
11 # User validates AI insight
12 await self.use_skill("feedback_capture",
    "capture_insight_validation",
13     user_id="user-uuid",
14     contact_id="contact-uuid",
15     insight_type="pain_point",
16     insight_text="Manual data entry consuming 60% of team time",
17     validation="confirmed",
18     confirmed_data={
19         "pain_point": "Team spends 15 hours/week on manual data
entry",
20         "exact_quote": "We're drowning in manual work",
21         "severity": "Critical",
22         "budget_allocated": True
23     }
24 )
25
26 # User adds custom fact
27 await self.use_skill("feedback_capture", "capture_custom_fact",
28     user_id="user-uuid",
29     contact_id="contact-uuid",
30     fact_type="goal",
31     fact_data={
32         "goal": "Launch new product by Q2 2025",
33         "timeline": "6 months",
34         "executive_mandate": True,
35         "source": "Told me in call today"
36     }
37 )
```

2.4 Other Shared Skills (Summary)